Smart X Part 3 Report

Network Topology and Mesh Routing

PROG7312 Portfolio of Evidence

Antonio Bechev
Student number ST10435330
EMERIS Sandton
13 November 2026

Report overview

Smart-X combines typed IoT telemetry, queued command processing and a simulated mesh dashboard in one .NET application. In Part 3 I extended the existing project with an AVL timestamp index, a weighted graph, breadth-first search and Prim's minimum spanning tree. The browser can locate a sensor, show its routes, simulate a failed node and retrieve its processed telemetry within a selected time range.

Contents

Report overview 1
Implementation report 3
Application purpose and architecture 3
Historical telemetry using an AVL tree 3
Network graph and availability 4
Breadth first search 5
Prim algorithm and the min heap 5
User interface and historical queries 6
Compile run and verify 7
Project completion report 8
Completion across the three parts 8
Integration challenges and resolutions 8
Learning and development practice 9
Current limitations and deployment boundaries 10
Lecturer feedback and improvements 10
Technology recommendations 11
Durable telemetry using PostgreSQL and Npgsql 11
MQTTnet and Home Assistant integration 11
Browser and container checks in continuous integration 11
Completion perspective 12

References..... 13

Report component	Assessment focus
Implementation report	Compilation and use, data structures, relevant code and efficiency
Project completion report	Work across the complete project, implementation challenges and learning
Technology recommendations	Justified improvements compatible with the .NET application

Source repository: https://github.com/ST10435330/ST10435330_POE_Part3_PROG7312

YouTube Demo: https://youtu.be/h3mjGzaj2D4

Implementation report

Application purpose and architecture

Managing many sensors creates two related problems: retrieving relevant historical readings and understanding whether a device can reach the gateway. A flat collection makes narrow timestamp searches expensive, while a device list alone does not explain intermediate connections or alternate routes. The Part 3 design gives historical data an ordered index and represents connectivity explicitly as a graph.

The solution targets net10.0. ASP.NET Core serves the Minimal API and the HTML, CSS and JavaScript client from wwwroot. Gateway manages profiles and typed raw histories. RealtimeEngine admits and processes queued work and maintains the indexed stream log. NetworkTopology translates registered sensor identities and current health into the graph used by the routing endpoints. Browser modules fetch these endpoints and update the three application views.

The device simulator sends real HTTP requests to the typed receiver endpoints. Accepted packets enter normal FIFO or critical priority queues and are processed by a hosted worker. Successful processing updates raw storage and the AVL stream timeline. A 202 response means admission to a queue; the receipt must reach Processed before storage is confirmed.

Module	Main responsibilities
Sensor ingestion	Registration, typed values, hierarchical location validation, raw history, attachments and meter aggregation
Command stream	FIFO and critical scheduling, receipts, live state, manual commands, stack undo and recommendations
Network topology	Mesh health, graph display, BFS and MST routes, failure controls and paged timestamp history

Historical telemetry using an AVL tree

The custom AvlTree<TKey, TValue> is a binary search tree that maintains a height balance at every node. AVL balancing keeps subtree heights within one level, supporting logarithmic search and updates (OpenDSA, n.d.). In this application, each sensor has a separate AvlTree<DateTimeOffset, StreamLog>, and a dictionary maps its GUID to that tree. Equal timestamps from different sensors therefore remain separate.

Insertion compares the incoming timestamp with the current key, descends left or right, then updates heights on the way back. A balance factor outside minus one to plus one causes a rotation. The implementation handles left-left and right-right cases with one rotation and the two mixed cases with a child rotation followed by a parent rotation. This is particularly useful when samples arrive in chronological order.

The following excerpt from CollectRange shows how a date query prunes branches. The comparisons include readings exactly at either boundary. The code returns matches in timestamp order.

```
int fromComparison = from.CompareTo(node.Key);
int toComparison = to.CompareTo(node.Key);
if (fromComparison < 0)
    CollectRange(node.Left, from, to, results);
if (fromComparison <= 0 && toComparison >= 0)
    results.Add(node.Value);
if (toComparison > 0)
    CollectRange(node.Right, from, to, results);
```

For n retained records and k matches, this range traversal takes O(log n + k). It still has to return every matching item, so a query covering the whole retained history cannot avoid that output cost. Insert, ContainsKey and oldest-key removal each take O(log n). These are algorithmic bounds rather than measured network-throughput results.

StreamHistory inserts a log only after a packet is successfully stored. When a timeline exceeds 2,048 entries, RemoveMinimum deletes its oldest timestamp and rebalances the remaining tree. Duplicate rejection continues at the ingestion layer. The generic tree can replace an existing key, but queue admission checks prevent a timestamp replay from becoming an additional stored stream entry.

```
timeline.Insert(work.Timestamp, row);
while (timeline.Count > 2048)
{
    timeline.RemoveMinimum();
}
```

Original History<T> lists are retained for Part 1 raw readings, preload samples and manual actuator-state updates. The AVL timeline has a narrower documented scope: successfully processed ingestion-queue telemetry since startup. Keeping those scopes explicit avoids presenting generated seed data as live HTTP traffic.

Network graph and availability

MeshGraph uses Dictionary<string, Dictionary<string, int>> as an adjacency list. The outer dictionary resolves an identifier and the inner dictionary stores its neighbours and link costs. Case-insensitive keys support registered identifiers and MAC addresses. Connect writes both directions, rejects unknown endpoints and self-links, and requires a positive cost.

```
adjacency[first][second] = cost;
adjacency[second][first] = cost;
```

Vertices correspond to registered sensors and CENTRAL GATEWAY. The generator connects each sensor to its preceding one and preceding two neighbours, where present. The first three sensors and every twenty-fifth sensor also receive a direct gateway link. Costs are generated integer units, rather than measured radio latency, signal strength or energy consumption.

Configured edges remain available in the model while a separate set identifies offline nodes. Before routing, NetworkTopology excludes sensors with a simulated failure, paused sensors and sensors whose telemetry status is Disconnected. The normal timeout is three expected reporting intervals. Startup seeds establish an initial reception time, and new registrations have an initial reporting grace period.

A node can be online but unreachable if all of its paths to the gateway are blocked. The API reports that state separately from an offline node. Clearing a simulated fault restores eligibility, while pause and freshness

conditions continue to apply. The graph is rebuilt from current profiles and health for each request, so new registrations and current availability are included.

Breadth first search

Traverse uses a FIFO queue and visited set to discover the online component connected to its starting node. FindPath also records a predecessor when a neighbour is first discovered. The destination path is reconstructed by following those predecessors back to the source and reversing the list.

```
previous.Add(neighbour, current);
queue.Enqueue(neighbour);
```

BFS expands nodes in hop layers. Its first route to the destination therefore uses the fewest edges, regardless of their weights. A full traversal takes O(V + E) time and O(V) auxiliary space for V vertices and E links. Offline nodes are skipped, and an unavailable source or disconnected destination produces an empty route.

This supports both the dashboard reachability information and its BFS route option. When an intermediate sensor is failed, the next calculation can discover another sequence of online neighbours. The failure tests also cover the case where every available path is blocked.

Prim algorithm and the min heap

The MST implementation starts at the gateway and grows its reachable tree by repeatedly choosing a minimum-cost candidate edge. This is the lazy form of Prim: obsolete candidates remain in the queue until they are removed and found to lead to an already visited node (Sedgewick and Wayne, n.d.).

The frontier is a PriorityQueue<MeshLink, int>. Microsoft documents that PriorityQueue removes the lowest priority value first and does not promise FIFO order for equal priorities (Microsoft, n.d.-a). Supplying link cost as the priority makes it a min-heap for Prim. Equal-cost alternatives can produce different valid trees.

```
candidates.Enqueue(new MeshLink(from, destination, cost), cost);
while (candidates.TryDequeue(out var link, out _))
{
    if (!visited.Add(link.To))
        continue;
    selectedLinks.Add(link);
    AddCandidates(link.To);
}
```

Each accepted edge adds one previously unvisited vertex, preventing cycles. A reachable component with Vc vertices has Vc minus one tree edges. Lazy Prim has O(E log E) edge-processing time and O(V + E) auxiliary storage in this implementation, including vertex bookkeeping. Total cost is summed as long so a large collection of integer weights does not overflow an int total.

A disconnected network has no single spanning tree covering all its vertices. MeshTreeResult therefore reports the gateway component, unreachable online nodes and offline nodes. The route service forms a To-to-From parent map from the selected links and follows it from the selected sensor back to the root.

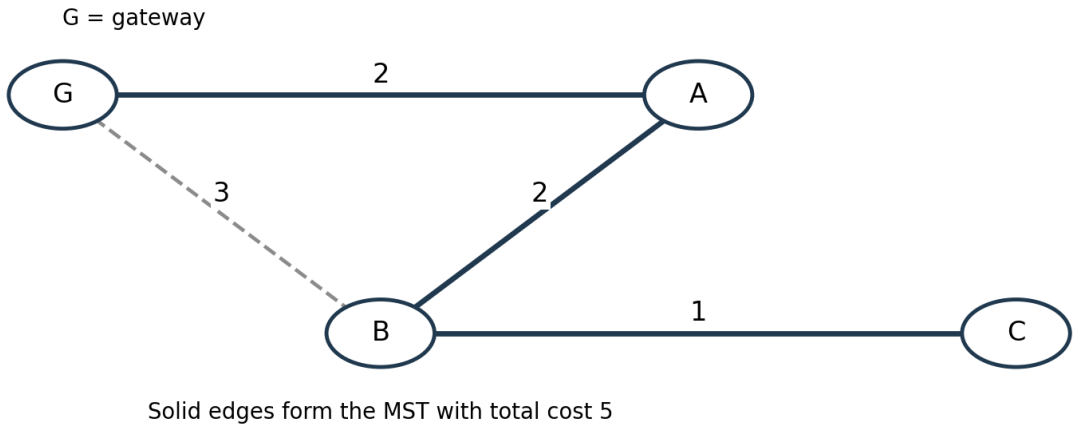


Figure 1 Illustrative weighted mesh used to explain the difference between total tree cost and an individual route cost.

In Figure 1, the MST has total cost 5 through G-A, A-B and B-C. B reaches G through A at cost 4, although its direct edge costs 3. This demonstrates why an MST does not guarantee each sensor an individually cheapest route. The application labels the modes BFS fewest hops and MST tree route accordingly.

User interface and historical queries

The topology page provides identifier search, paged sensor selection, full-network metrics, a bounded graph window, route-mode selection and failure controls. It normally renders up to 40 nodes including the selected route and nearby nodes, while server-side routing and metrics use the complete graph. A selected sensor outside the initial diagram window remains searchable and is brought into view.

The telemetry panel follows the selected sensor and submits local datetime inputs as UTC ISO timestamps. Either date bound can be omitted. The AVL query selects all retained matches before paging backwards from the newest end. The API preserves ascending order inside each page for the existing command-stream chart; the topology table reverses the page for display.

The response distinguishes stored, matched and shown counts, and supplies older/newer page flags. Pages contain 1 to 500 readings. The latest page refreshes every three seconds while visible; older pages remain still until refreshed or changed. Request serial numbers prevent a late response for an earlier selection from replacing the final selected sensor's rows. Empty ranges and invalid bounds have explicit messages.

Operation	Principal structure	Cost or bound
Timestamp insertion or oldest removal	AVL tree	O(log n)
Inclusive timestamp query	AVL tree	O(log n + k)
Stored per sensor stream history	AVL nodes	At most 2,048 entries
Reachability and fewest hops	BFS queue and visited map	O(V + E)
Gateway component MST	Adjacency list and candidate heap	O(V + E log E), including bookkeeping
Configured graph storage	Adjacency dictionaries	O(V + E)

Compile run and verify

The complete source is stored in the Part 3 GitHub repository. From its root, restore and compile the solution with the .NET 10 SDK, then start the API. The browser is served by the same process, with no separate npm build or external database account required.

```
dotnet restore SmartX.sln -m:1
dotnet build SmartX.sln -c Release -m:1
dotnet run --project src/SmartX.Api -c Release --no-build `
  --urls http://localhost:5080
```

Open http://localhost:5080. For a demonstration, start the 20-sensor simulator, select ESP32-0004, compare its route modes, fail a relay and clear the fault. The demo sends for the first 20 registered sensors; other profiles expire unless they transmit. A 60-reading command-stream batch can provide enough history to demonstrate 25-row pages after normal processing completes.

```
dotnet run --project tests/SmartX.Checks -c Release --no-build
python scripts/http_checks.py
```

The GitHub Actions result for 00f4b9e was confirmed green on 12 November 2026. That workflow builds both projects, executes the C# checks, syntax-checks all six client JavaScript files and runs the real HTTP suite against an isolated temporary server. It covers routing failures, recovery, MAC lookup, freshness timeout and history paging alongside earlier telemetry and command scenarios. Exact assertion totals and performance figures are not claimed without their captured output.

A Dockerfile and Compose configuration are included. The container maps localhost port 5080 to port 8080 and retains profiles and attachments in a named volume. A final Docker runtime pass and final browser interaction/layout record remain separate verification tasks; a green build and HTTP run does not itself establish those results.

Project completion report

Completion across the three parts

Part 1 established the gateway data model and ingestion interface. Generic telemetry packets preserve float moisture values, integer wattage and boolean actuator states. Jagged batches, bounded lists, recursive deployment-path validation and operator-overloaded meter readings satisfy the earlier programming requirements. The fresh installation creates 1,000 sensor profiles and 120 preload samples per profile, which are explicitly separate from live API traffic.

Part 2 added asynchronous HTTP admission, a normal FIFO queue and a separately bounded critical queue, receipt tracking and a background worker. Dictionaries provide device identity lookup and sets track unique alert states. Commands save prior actuator states in a stack, including grouped shutdown restoration. Suggestions use configured pump relationships, frequency counts and a conditional moisture trend; they still require a user decision.

Part 3 integrates the tree, heap and graph requirements into the existing application rather than presenting isolated algorithm examples. All menu views are active. The new routing service uses registered sensor identities, the browser calls its real endpoints, and the history panel uses the AVL-backed endpoint. Failure controls change the routing calculation, while stored readings remain available for investigation.

Project area	Completed behaviour	Verification basis
Part 1 foundation	Typed ingestion, profiles, recursion, meter operations and attachments	C# and HTTP checks
Part 2 control	Queue scheduling, live registry, commands, undo and suggestions	C# and HTTP checks
Part 3 analysis	AVL history, graph, BFS, Prim, routing controls and history UI	Algorithm checks, HTTP checks and JavaScript syntax checks
Packaging	One hosted client, repository, README, API notes and Docker files	Source/configuration inspection; Docker runtime record separate

Integration challenges and resolutions

One challenge was adding the topology and history features without disrupting the earlier ingestion and command workflows. I learned that the network diagram needed to use the same sensor identities and backend data as the rest of Smart-X. Connecting the sensor registry to the graph model, route endpoints and dashboard controls allowed the selected sensor's status and paths to appear together.

I also needed to understand how to represent the branches between sensors. I learned that these branches are graph edges, with valid endpoints, a connection cost and support for communication in both directions. The full mesh can contain cycles and alternative paths, while the minimum spanning tree connects the reachable nodes without cycles. This helped me understand why the graph and its spanning tree are different views of the same network.

Another challenge was understanding the different purposes of BFS and Prim's algorithm. I learned that BFS uses a queue to find the fewest-hop route, while Prim uses a min-heap priority queue to build a tree with the lowest total connection cost. The dashboard therefore provides separate routing options. I also learned that a route through the MST is not guaranteed to be the cheapest individual route, and that devices outside the gateway's reachable component must be reported separately.

The failure controls showed me that recent telemetry, mesh availability and gateway reachability are different states. A sensor can have a recent reading while being deliberately failed in the mesh model. Clearing its failure does not make stale telemetry current. Keeping these statuses separate made the behaviour clearer, while recalculating the routes allowed the application to use alternative connections where available. Previously processed history remained readable during failure.

Replacing the timestamp dictionary with an AVL tree caused compiler errors because TelemetryInbox still required ContainsKey and TrendAnalysis still required Values. Adding compatible tree-backed members resolved those dependencies. Another error came from using avl in a check before declaring and populating it. I learned to check how a data structure is used throughout the application and to prepare test data before checking removal or retention behaviour.

Telemetry history also needed to preserve sample order when critical packets were processed ahead of older normal packets. I learned why sample time and processing time must be recorded separately. The AVL index orders readings by their sample timestamps, while the processing metadata shows when they were handled. Connecting this index to the history endpoint allowed the table to support date searches and older and newer pages.

The date picker required consistent handling of local times and UTC timestamps. I learned how the selected dates become API parameters and why both the interface and backend need to reject an inverted range. The query includes both boundaries. Checking the request version and selected sensor prevents an older response from displaying the wrong history, while keeping older pages still makes the records easier to inspect.

Finally, I learned that verification needed to cover the integrated application. The workflow was updated to include the topology JavaScript and HTTP checks alongside the existing build and data structure checks. Reviewing the successful run helped me understand what had been verified. I also reviewed the documentation so its instructions and explanations matched the application, while recognizing that browser and container checks require their own evidence.

Learning and development practice

Part 3 helped me understand how the data structures support a practical feature rather than existing as separate classroom examples. I learned how to model the Smart-X sensors as graph vertices and their connections as weighted edges. I now understand why a mesh can offer alternative routes, how a failed relay affects reachability, and why the visual diagram must reflect the underlying graph state.

I developed a clearer understanding of BFS by following how its queue explores devices in layers and how the recorded predecessors reconstruct a route. This connected the algorithm to a visible result: the fewest-hop path from a selected sensor to the gateway. I also learned why fewer hops do not necessarily mean lower total link cost.

Working through Prim's algorithm helped me understand the role of a min-heap and the reason for checking whether a candidate device has already been reached. I learned that the algorithm minimizes the total cost of the tree across the reachable component. The distinction between a full mesh and a tree became clearer: the

mesh can contain cycles, while the chosen tree avoids them. A tree connecting a given number of nodes has one fewer connection than nodes, which provides a useful way to check the result.

The mathematical explanations helped me relate these ideas to the scenario. I learned to calculate hop counts and total connection costs, interpret reachability, and explain algorithm efficiency in practical terms. I also learned to qualify those results correctly. The generated costs support a routing simulation; they do not establish actual wireless speed or power consumption.

The telemetry history work improved my understanding of balanced trees. I learned why readings are indexed by timestamp, how AVL rotations keep the tree balanced, and how a range query avoids searching branches outside the requested period. I also learned that the number of returned records contributes to the work performed by the query, and that removing old readings must preserve the tree's balance.

Working through the history panel and date picker helped me follow the complete path from a browser control to a backend result. I learned how local date selections become UTC request values, why validation belongs in both the interface and API, and how date filters interact with paging. Handling empty results and late responses also showed me that correct data structures alone do not guarantee a reliable interface.

Resolving the compiler errors improved my approach to debugging.

The documentation work also developed my technical communication. Reviewing the README, API notes, data structure explanations, verification record, main report, feedback updates, declaration and demonstration script helped me explain the features, algorithm choices and limitations. I learned to separate implemented features from recommendations and to support verification claims with the relevant evidence.

Applying the changes helped me connect the theory to my existing Smart-X program. I remain responsible for understanding and accurately presenting the final submission.

Current limitations and deployment boundaries

The application is a single-process simulation with a registration limit of 5,000 sensors. Telemetry, queues, undo history, learned associations and simulated failures are volatile. Profiles and attachments persist, but a restart regenerates preload data and resets the live timeline. Large numbers of per-sensor histories can still require substantial memory despite the individual bounds.

Mesh costs and connections are generated. The routing API models eligible paths, while telemetry continues through HTTP; it does not control ESP32 radios or prove energy savings. Device authentication, durable distributed queues and real actuator acknowledgements are further deployment requirements. Strict critical priority can delay normal work under sustained urgent traffic.

AI assistance was used for code examples, troubleshooting, test guidance and documentation drafting during development. The source, test results and demonstration evidence provide a basis for reviewing the resulting work. This report does not claim physical hardware validation or an unrecorded Docker/browser pass.

Lecturer feedback and improvements

No specific corrective lecturer feedback was received for Parts 1 and 2. The Part 3 enhancements therefore respond to the final brief and development testing. A separate feedback-updates file records this explicitly and identifies the implemented improvements without attributing them to lecturer instructions that were not provided.

Technology recommendations

The following recommendations address limitations visible in the current implementation. They are proposed extensions, rather than features already implemented or benchmarked. Their priority depends on whether Smart-X remains a local assessment application or becomes a service used with real devices.

Durable telemetry using PostgreSQL and Npgsql

The first functional extension should be persistent telemetry. The current AVL timeline is bounded and resets when the process restarts. PostgreSQL accessed through Npgsql would allow historical data to survive restarts and support retention beyond the in-memory window. Npgsql provides an Entity Framework Core provider and documents ASP.NET dependency-injection configuration, which fits the existing C# service structure (Npgsql, n.d.).

I would store sensor identity, sample time, typed value/category and processing metadata, with an index beginning with sensor ID and timestamp. The existing AVL tree could remain a recent-data index or teaching implementation while the database serves longer periods. Migration would require an explicit duplicate key policy, UTC conventions, backup/retention rules and provider versions matched to the chosen EF Core target. The expected benefit is durability and a clearer scaling path; a throughput gain would need measurement.

MQTTnet and Home Assistant integration

For physical device integration, an MQTT bridge could complement the HTTP receiver. MQTTnet supplies .NET MQTT client and broker functionality, while Home Assistant supports MQTT-based device/entity discovery (MQTTnet Contributors, n.d.; Home Assistant, n.d.). A C# bridge can therefore translate device messages into the existing typed work items and preserve validation, receipt handling and queue priorities.

This would reduce coupling between device publishing and the browser/API request pattern, and could expose moisture readings or actuator availability to a Home Assistant dashboard. Topic naming, credentials, transport protection, duplicate-delivery handling and actuator acknowledgements would need explicit design. MQTT would provide messaging integration; it would not itself replace the radio mesh model or demonstrate that the simulated MST controls physical routing.

Browser and container checks in continuous integration

The confirmed workflow verifies compilation, C# logic, JavaScript syntax and real HTTP behaviour. Extending Playwright coverage to the new topology view would check the remaining interaction boundary. Playwright documents GitHub Actions execution and test-report publication, making it suitable for the existing repository workflow (Microsoft, n.d.-c).

The browser cases should cover route-mode selection, relay failure, history dates, paging, rapid selection and a narrow viewport. A separate Docker build/health smoke check would verify the included container configuration rather than inferring it from a native .NET run. Both improvements use the existing source and endpoint contracts and would provide more direct release evidence.

Priority	Proposed extension	Reason in this project
Immediate verification	Part 3 Playwright and Docker smoke checks	Close the remaining UI and container evidence gaps
First functional extension	PostgreSQL through Npgsql	Preserve telemetry beyond a restart and define longer retention
Performance work	OpenTelemetry instrumentation	Measure queue/routing behaviour before optimising it
Hardware integration	MQTTnet bridge and Home Assistant	Connect real publishers and external dashboards through the validated pipeline

Completion perspective

Smart-X now combines the required historical index and mesh algorithms with the earlier ingestion and command workflows. The main contribution of Part 3 is the connection between those structures and observable application behaviour: date queries, reachability, route selection and failure recovery. Persistent storage, measured observability and additional integration tests provide practical next steps without overstating the capabilities of the current simulation.

References

Bechev, A. (2026). Smart-X Part 3 source code. Verified CI baseline 00f4b9e. Available at: https://github.com/ST10435330/ST10435330_POE_Part3_PROG7312 (Accessed: 13 November 2026).

Home Assistant (n.d.). MQTT integration. Available at: https://www.home-assistant.io/integrations/mqtt/ (Accessed: 13 November 2026).

Microsoft (n.d.-a). PriorityQueue TElement TPriority class. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.priorityqueue-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (n.d.-b). .NET observability with OpenTelemetry. Available at: https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel (Accessed: 13 November 2026).

Microsoft (n.d.-c). Playwright setting up CI. Available at: https://playwright.dev/docs/ci-intro (Accessed: 13 November 2026).

MQTTnet Contributors (n.d.). MQTTnet project documentation. Available at: https://github.com/dotnet/MQTTnet (Accessed: 13 November 2026).

Npgsql (n.d.). Npgsql Entity Framework Core provider. Available at: https://www.npgsql.org/efcore/ (Accessed: 13 November 2026).

OpenDSA (n.d.). The AVL Tree. Available at: https://opendsa-server.cs.vt.edu/ODSA/Books/duke/cs201/spring-2025/MW/html/AVL.html (Accessed: 13 November 2026).

Sedgewick, R. and Wayne, K. (n.d.). Minimum spanning trees. Algorithms fourth edition online material. Available at: https://algs4.cs.princeton.edu/43mst/ (Accessed: 13 November 2026).